Lesson 66: Neural Tangent Kernel and Rademacher Complexity

AI/ML Mathematics

Contents

1 Motivation and Overview **2**

2 References **3**

3 Rademacher Complexity **3**

4 Rademacher Complexity of Neural Networks **5**
- 4.1 The Contraction Lemma . . . 5
- 4.2 Spectrally-Normalized Bound . . . 6

5 The Neural Tangent Kernel **7**
- 5.1 Definition and Motivation . . . 7
- 5.2 NTK Dynamics of Gradient Descent . . . 8
- 5.3 The Infinite-Width Limit . . . 9
- 5.4 Lazy Training: NTK Stays Constant . . . 9

6 Implications for Generalization **10**
- 6.1 NTK Generalization via Kernel Theory . . . 10
- 6.2 Limitations of the NTK Framework . . . 11

7 Exercises **12**

8 Summary **13**

9 Further Reading **14**

1 Motivation and Overview

Modern neural networks used in deep RL and supervised learning are massively overparameterized: the number of parameters p vastly exceeds the number of training examples n. Classical learning theory (Lesson 59) predicts that such models should overfit catastrophically—the VC dimension of a network with p parameters is $\Theta(p)$, giving a vacuous generalization bound when $p \gg n$. Yet in practice, overparameterized networks trained by gradient descent generalize remarkably well. This is the *generalization puzzle* of deep learning.

Two complementary theoretical frameworks resolve (in part) this puzzle. First, *Rademacher complexity bounds* based on weight norms rather than parameter counts show that generalization depends on how "small" the learned weights are, not on how many there are. The spectrally-normalized bound of Bartlett et al. [2017] shows that the generalization gap scales with the product of spectral norms $\prod_\ell \|W_\ell\|_2$ divided by $\sqrt{n}$—a quantity that can be small even for heavily overparameterized networks. Second, the *neural tangent kernel* (NTK) of Jacot et al. [2018] shows that in the infinite-width limit, neural network training by gradient descent is equivalent to kernel regression with a fixed, deterministic kernel. This connects deep learning to the well-understood theory of RKHS (Lesson 62), providing both optimization and generalization guarantees. The tools developed in this lesson will let us understand why overparameterized networks generalize, how weight norms control capacity, and what regime of training the NTK theory describes.

Why NTK and Rademacher Complexity Matter for RL/ML

- **Generalization in deep RL.** Policy and value networks in PPO, SAC, and DQN are typically overparameterized. Norm-based Rademacher bounds explain why these networks generalize from limited environment interactions despite having millions of parameters.
- **Implicit regularization.** Gradient descent on overparameterized networks converges to minimum-norm solutions (in the NTK regime), providing implicit regularization that controls generalization without explicit weight decay.
- **Neural network as kernel method.** The NTK framework allows transferring kernel ridge regression theory (Lesson 62) to neural networks, yielding concrete convergence and generalization guarantees.
- **Architecture selection.** Understanding that spectral norms—not parameter counts—control generalization informs the choice of network architecture and initialization scheme in deep RL.

Prerequisites. Rademacher complexity and its connection to generalization (Lesson 59, Definition 5.1). The approximation-estimation tradeoff (Lesson 60, Section 3). RKHS and kernel ridge regression (Lesson 62, Sections 3–5). Spectral norm and operator norm of matrices (Lesson 8, Definition 3.1). Gradient descent and convex optimization (Lesson 30). Deep neural networks and their function classes (Lesson 65, Definition 3.1).

Learning objectives. After completing this lesson, you will be able to:

- **Define** Rademacher complexity and **prove** the fundamental generalization bound relating it to the uniform deviation of empirical risk from population risk.
- **State and prove** the spectrally-normalized Rademacher complexity bound for deep networks, and **explain** why it depends on weight norms rather than parameter counts.
- **Define** the neural tangent kernel and **derive** the linearized dynamics of gradient descent in the infinite-width limit.

- **Prove** that the NTK converges to a deterministic kernel at initialization and remains approximately constant during training (the lazy training regime).
- **Apply** NTK theory to deduce generalization bounds for overparameterized networks via kernel regression theory.
- **Evaluate** the limitations of the NTK framework, including its failure to capture feature learning.

2 References

- Bartlett and Mendelson [2002] — Foundational paper on Rademacher complexity: definitions, structural results, and the generalization bound.
- Bartlett et al. [2017] — Spectrally-normalized margin bounds for deep networks; the key norm-based Rademacher complexity result.
- Jacot et al. [2018] — The neural tangent kernel: convergence and generalization in the infinite-width limit.
- Lee et al. [2019] — Wide neural networks evolve as linear models under gradient descent; empirical validation of the NTK regime.
- Arora et al. [2019] — Fine-grained analysis of optimization and generalization for overparameterized two-layer networks.
- Du et al. [2019] — Gradient descent provably optimizes overparameterized neural networks: convergence guarantees via the NTK.

3 Rademacher Complexity

Rademacher complexity is a data-dependent measure of the richness of a function class. It refines the VC dimension by accounting for the specific distribution of the data and the real-valued nature of the functions (rather than just binary labelings).

Definition 3.1 (Rademacher Random Variables)**.** A *Rademacher random variable* ε takes values $+1$ and -1 each with probability $1/2$. Rademacher random variables are symmetric: $\varepsilon \overset{d}{=} -\varepsilon$.

Definition 3.2 (Empirical Rademacher Complexity)**.** Let $\mathcal{F}$ be a class of functions $f\colon \mathcal{X} \to \mathbb{R}$ and let $S = (x_1, \ldots, x_n) \in \mathcal{X}^n$ be a fixed sample. The *empirical Rademacher complexity* of $\mathcal{F}$ with respect to S is

$$\hat{\mathrm{Rad}}_S(\mathcal{F}) = \mathbb{E}_\varepsilon\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^n \varepsilon_i f(x_i)\right],$$

where $\varepsilon_1, \ldots, \varepsilon_n$ are i.i.d. Rademacher random variables.

Definition 3.3 (Rademacher Complexity)**.** The *Rademacher complexity* of $\mathcal{F}$ with respect to a distribution P on $\mathcal{X}$ and sample size n is

$$\mathrm{Rad}_n(\mathcal{F}) = \mathbb{E}_{S\sim P^n}\left[\hat{\mathrm{Rad}}_S(\mathcal{F})\right] = \mathbb{E}_{S,\varepsilon}\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^n \varepsilon_i f(x_i)\right].$$

Remark 3.4. The Rademacher complexity measures how well the function class $\mathcal{F}$ can *correlate with random noise*. If $\mathcal{F}$ is rich enough to fit arbitrary label assignments $(\varepsilon_1, \ldots, \varepsilon_n) \in \{-1, +1\}^n$, then $\hat{\mathrm{Rad}}_S(\mathcal{F})$ is close to 1. If $\mathcal{F}$ consists only of constant functions, then $\hat{\mathrm{Rad}}_S(\mathcal{F}) = 0$. In general, $0 \le \hat{\mathrm{Rad}}_S(\mathcal{F}) \le \sup_{f\in\mathcal{F}} \|f\|_\infty$.

Example 3.5 (Rademacher Complexity of Linear Classifiers)**.** Let $\mathcal{F} = \{x \mapsto w^\top x : \|w\|_2 \le B\}$ and suppose $\|x_i\|_2 \le C$ for all i. Then

$$\begin{aligned}\hat{\mathrm{Rad}}_S(\mathcal{F}) &= \mathbb{E}_\varepsilon\left[\sup_{\|w\|_2\le B} \frac{1}{n} w^\top \sum_{i=1}^n \varepsilon_i x_i\right] = \frac{B}{n}\mathbb{E}_\varepsilon\left[\left\|\sum_{i=1}^n \varepsilon_i x_i\right\|_2\right] \\ &\le \frac{B}{n}\sqrt{\mathbb{E}\left[\left\|\sum_{i=1}^n \varepsilon_i x_i\right\|_2^2\right]} = \frac{B}{n}\sqrt{\sum_{i=1}^n \|x_i\|_2^2} \le \frac{BC}{\sqrt{n}},\end{aligned}$$

where we used Jensen's inequality and the fact that $\mathbb{E}[\varepsilon_i\varepsilon_j] = \delta_{ij}$. Thus $\mathrm{Rad}_n(\mathcal{F}) \le BC/\sqrt{n}$, which is independent of the dimension d of the weight vector.

We now prove the fundamental generalization bound via Rademacher complexity.

Theorem 3.6 (Rademacher Generalization Bound)**.** *Let $\mathcal{F}$ be a class of functions $f\colon \mathcal{X} \to [0,1]$, and let $(X_1,Y_1),\dots,(X_n,Y_n)$ be i.i.d. from a distribution P on $\mathcal{X}\times\mathcal{Y}$. Define the loss class $\mathcal{G} = \{(x,y) \mapsto \ell(f(x),y) : f \in \mathcal{F}\}$ where ℓ takes values in $[0,1]$. Then for any $\delta > 0$, with probability at least $1-\delta$ over the sample S:*

$$\sup_{f\in\mathcal{F}} \big|R(f) - \hat{R}_n(f)\big| \le 2\mathrm{Rad}_n(\mathcal{G}) + \sqrt{\frac{\ln(2/\delta)}{2n}}.$$

If the loss ℓ is ρ-Lipschitz in its first argument, then $\mathrm{Rad}_n(\mathcal{G}) \le \rho\,\mathrm{Rad}_n(\mathcal{F})$.

Proof. **Step 1: Symmetrization.** Define $\Phi(S) = \sup_{g\in\mathcal{G}}\big[\mathbb{E}[g(X,Y)] - \frac{1}{n}\sum_{i=1}^n g(X_i,Y_i)\big]$. We first show $\mathbb{E}[\Phi(S)] \le 2\mathrm{Rad}_n(\mathcal{G})$.

Let $S' = (X_1',Y_1'),\dots,(X_n',Y_n')$ be an independent copy of S. For each $g\in\mathcal{G}$, $\mathbb{E}[g(X_j',Y_j')] = \mathbb{E}[g(X,Y)]$, so

$$\begin{aligned}\mathbb{E}[\Phi(S)] &= \mathbb{E}_S\left[\sup_{g\in\mathcal{G}} \mathbb{E}_{S'}\left[\frac{1}{n}\sum_{i=1}^n \big(g(X_i',Y_i') - g(X_i,Y_i)\big)\right]\right] \\ &\le \mathbb{E}_{S,S'}\left[\sup_{g\in\mathcal{G}} \frac{1}{n}\sum_{i=1}^n \big(g(X_i',Y_i') - g(X_i,Y_i)\big)\right].\end{aligned}$$

Since $g(X_i',Y_i') - g(X_i,Y_i)$ is symmetric about 0 (the pairs (X_i,Y_i) and (X_i',Y_i') are exchangeable), we can introduce Rademacher variables ε_i:

$$\begin{aligned}\mathbb{E}[\Phi(S)] &\le \mathbb{E}_{S,S',\varepsilon}\left[\sup_{g\in\mathcal{G}} \frac{1}{n}\sum_{i=1}^n \varepsilon_i\big(g(X_i',Y_i') - g(X_i,Y_i)\big)\right] \\ &\le \mathbb{E}_{S',\varepsilon}\left[\sup_{g\in\mathcal{G}} \frac{1}{n}\sum_{i=1}^n \varepsilon_i g(X_i',Y_i')\right] + \mathbb{E}_{S,\varepsilon}\left[\sup_{g\in\mathcal{G}} \frac{1}{n}\sum_{i=1}^n (-\varepsilon_i) g(X_i,Y_i)\right] \\ &= 2\mathrm{Rad}_n(\mathcal{G}).\end{aligned}$$

Step 2: Concentration. The function $\Phi(S)$ changes by at most $1/n$ when a single sample (X_i,Y_i) is replaced (since g is bounded in $[0,1]$). By McDiarmid's inequality (a consequence of the bounded differences inequality, see Lesson 12):

$$\mathbb{P}\big(\Phi(S) - \mathbb{E}[\Phi(S)] \ge t\big) \le \exp\left(-\frac{2n^2t^2}{n\cdot(1/n)^2\cdot n}\right) = \exp(-2nt^2).$$

Setting $t = \sqrt{\ln(2/\delta)/(2n)}$ and combining with $\mathbb{E}[\Phi(S)] \le 2\mathrm{Rad}_n(\mathcal{G})$ gives the one-sided bound. The two-sided bound follows by applying the same argument to $-g$.

Step 3: Lipschitz contraction. If $\ell(\cdot, y)$ is ρ-Lipschitz for each y, then by the contraction lemma (Ledoux–Talagrand):

$$\mathbb{E}_\varepsilon\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \varepsilon_i \ell(f(x_i), y_i)\right] \le \rho\, \mathbb{E}_\varepsilon\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \varepsilon_i f(x_i)\right],$$

giving $\mathrm{Rad}_n(\mathcal{G}) \le \rho\, \mathrm{Rad}_n(\mathcal{F})$. □

Rademacher Generalization Bound

For any function class $\mathcal{F}$ with bounded loss, the uniform deviation between population risk and empirical risk satisfies

$$\sup_{f\in\mathcal{F}} \left|R(f) - \hat{R}_n(f)\right| \le 2\mathrm{Rad}_n(\mathcal{F}) + O\left(\sqrt{\frac{\log(1/\delta)}{n}}\right)$$

with high probability. The Rademacher complexity $\mathrm{Rad}_n(\mathcal{F})$ is a more refined measure of function class capacity than the VC dimension, as it accounts for the distribution of the data and the real-valued nature of the functions.

Example 3.7 (Rademacher vs. VC for Halfspaces)**.** For halfspace classifiers $\mathcal{F} = \{x \mapsto \mathrm{sign}(w^\top x) : \|w\|_2 \le B\}$ on data with $\|x\|_2 \le C$:

- **VC bound** (Lesson 59): generalization gap $O(\sqrt{d \log n/n})$, depending on dimension d.
- **Rademacher bound**: generalization gap $O(BC/\sqrt{n})$, independent of dimension d.

The Rademacher bound is tighter when BC is small (i.e., when the margin is large), which is precisely the regime where SVMs work well.

Having established the Rademacher framework, we now apply it to deep neural networks.

4 Rademacher Complexity of Neural Networks

The key result of this section is that the Rademacher complexity of deep networks depends on the *spectral norms* of the weight matrices, not on the number of parameters. This explains why overparameterized networks can generalize.

4.1 The Contraction Lemma

Lemma 4.1 (Contraction Lemma (Ledoux–Talagrand))**.** *Let $\mathcal{F}$ be a function class and let $\phi_1, \ldots, \phi_n \colon \mathbb{R} \to \mathbb{R}$ be ρ-Lipschitz functions with $\phi_i(0) = 0$. Then*

$$\mathbb{E}_\varepsilon\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \varepsilon_i \phi_i(f(x_i))\right] \le \rho\, \mathbb{E}_\varepsilon\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_{i=1}^{n} \varepsilon_i f(x_i)\right].$$

Proof. This is a classical result; see Ledoux and Talagrand [1991, Theorem 4.12]. The proof proceeds by induction on n. For $n = 1$: by the Lipschitz condition, $\sup_{t\in T} \varepsilon\phi(t) \le \rho \sup_{t\in T} \varepsilon t$ for any index set T (where $T = \{f(x_1) : f \in \mathcal{F}\}$). Taking expectations over ε gives the result. The inductive step conditions on $\varepsilon_1, \ldots, \varepsilon_{n-1}$ and applies the $n = 1$ case to ε_n. □

Remark 4.2. The ReLU activation $\sigma(t) = \max(0, t)$ is 1-Lipschitz with $\sigma(0) = 0$. Therefore, applying a ReLU layer does not increase the Rademacher complexity. This is the key reason why depth, per se, does not inflate the Rademacher complexity—only the weight norms matter.

4.2 Spectrally-Normalized Bound

Theorem 4.3 (Bartlett, Foster, and Telgarsky, 2017). *Consider an L-layer ReLU network (Definition 3.1 in Lesson 65): $f(x) = W_L\sigma(W_{L-1}\sigma(\cdots\sigma(W_1x)\cdots))$ with weight matrices $W_\ell \in \mathbb{R}^{n_\ell \times n_{\ell-1}}$ (biases omitted for clarity). Let $\mathcal{F}$ be the class of all such networks with $\|W_\ell\|_2 \le s_\ell$ (spectral norm bound) for each layer ℓ. If $\|x_i\|_2 \le B_x$ for all data points, then the empirical Rademacher complexity satisfies*

$$\hat{\mathrm{Rad}}_S(\mathcal{F}) \le \frac{B_x \cdot \prod_{\ell=1}^{L} s_\ell}{\sqrt{n}} \cdot O\left(\sqrt{L\log(W)}\right), \tag{1}$$

where $W = \max_\ell n_\ell$ is the maximum width.

Proof. We prove a simplified version of the bound, establishing the product structure; the logarithmic factors are refined in Bartlett et al. [2017].

Step 1: Peel off the output layer. Define $h(x) = \sigma(W_{L-1}\sigma(\cdots\sigma(W_1x)\cdots))$ so that $f(x) = W_Lh(x)$. The Rademacher complexity of the composition with the linear map W_L satisfies

$$\hat{\mathrm{Rad}}_S(\{x \mapsto W_Lh(x)\}) \le \|W_L\|_2 \cdot \hat{\mathrm{Rad}}_S(\{x \mapsto h(x)\}),$$

by viewing W_L as a linear operator and using the definition of the spectral norm. More precisely, for scalar output ($n_L = 1$), W_L is a row vector with $\|W_L\|_2 = \|W_L\|_F$, and

$$\begin{aligned}\mathbb{E}_\varepsilon\left[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_i \varepsilon_i W_L h(x_i)\right] &= \mathbb{E}_\varepsilon\left[\sup_{\|W_L\|_2\le s_L} W_L \cdot \sup_{h\in\mathcal{H}_{L-1}} \frac{1}{n}\sum_i \varepsilon_i h(x_i)\right] \\ &\le s_L \cdot \mathbb{E}_\varepsilon\left[\sup_{h\in\mathcal{H}_{L-1}} \frac{1}{n}\left\|\sum_i \varepsilon_i h(x_i)\right\|_2\right],\end{aligned}$$

where $\mathcal{H}_{L-1}$ is the class of functions computed by the first $L-1$ layers.

Step 2: Peel off the activation. The ReLU activation is 1-Lipschitz with $\sigma(0) = 0$. By the contraction lemma (Lemma 4.1), replacing $\sigma(W_\ell\cdot)$ with $W_\ell\cdot$ does not increase the Rademacher complexity (applied coordinatewise).

Step 3: Iterate. Repeating Steps 1 and 2 for layers $L-1, L-2, \ldots, 1$, each peeling contributes a factor of $s_\ell = \|W_\ell\|_2$. The base case is the input layer, where the Rademacher complexity of $\{x \mapsto x\}$ with respect to data bounded by B_x is $B_x/\sqrt{n}$ (by Example 3.5 with $B = 1$).

Step 4: Combine. The product of the spectral norm factors gives

$$\hat{\mathrm{Rad}}_S(\mathcal{F}) \le \frac{B_x \cdot \prod_{\ell=1}^{L} s_\ell}{\sqrt{n}} \cdot C(L, W),$$

where $C(L, W)$ accounts for the vector-valued nature of the intermediate layers (the Rademacher complexity of vector-valued functions picks up a factor of $\sqrt{\log W}$ per layer from covering number arguments). Careful analysis in Bartlett et al. [2017] shows $C(L, W) = O(\sqrt{L\log W})$. □

Norm-Based Generalization for Deep Networks

The generalization gap of a deep ReLU network is controlled by

$$\text{Generalization gap} \le O\left(\frac{B_x \cdot \prod_{\ell=1}^{L} \|W_\ell\|_2 \cdot \sqrt{L\log W}}{\sqrt{n}}\right),$$

which depends on the **spectral norms** of the weight matrices and the **data norm** B_x, but is **independent of the number of parameters**. An overparameterized network with small spectral norms can generalize well even when the number of parameters far

exceeds the sample size.

Example 4.4 (Parameter-Free Bound for a DQN)**.** Consider a DQN with $L = 5$ layers, width $W = 256$, and spectral norms $\|W_\ell\|_2 \le 2$ for each layer. The total parameter count is roughly $5 \times 256^2 \approx 327{,}680$. With $n = 10{,}000$ training transitions and $B_x = 1$:

- **VC bound:** $O(\sqrt{327{,}680/10{,}000}) \approx 5.7$ — vacuous (the bound exceeds 1).
- **Spectral bound:** $O(2^5 \cdot \sqrt{5 \log 256}/\sqrt{10{,}000}) = O(32 \cdot 3.1/100) \approx 1.0$ — borderline, but dramatically tighter than the VC bound.

With spectral normalization or weight decay to keep $\|W_\ell\|_2$ close to 1, the bound improves to $O(0.03)$—a meaningful guarantee.

Spectral Normalization in Deep RL

The spectral norm bound motivates the use of *spectral normalization* in deep RL:

- In DDPG and TD3, spectral normalization of the critic network stabilizes training and improves generalization.
- The bound $\prod_\ell \|W_\ell\|_2$ shows that controlling the product of spectral norms (not the individual norms) is what matters for generalization.
- Weight decay (ℓ^2 regularization) implicitly controls spectral norms, providing a connection between the common practice of regularization and the theoretical bound.

Norm-Based Bounds Can Still Be Loose

While spectrally-normalized bounds are much tighter than VC bounds for overparameterized networks, they are still often *vacuous* for practical networks (the bound exceeds 1 for classification). The product $\prod_\ell \|W_\ell\|_2$ grows exponentially with depth unless each spectral norm is close to 1. Tighter bounds require margin-based analysis or PAC-Bayesian arguments; see Neyshabur et al. [2015] for further developments.

The norm-based bounds explain generalization in terms of weight properties. The next section provides a complementary perspective: the neural tangent kernel explains the *dynamics* of training and connects overparameterized networks to kernel methods.

5 The Neural Tangent Kernel

The neural tangent kernel (NTK) reveals that in the infinite-width limit, training a neural network by gradient descent is equivalent to performing kernel regression with a specific, deterministic kernel.

5.1 Definition and Motivation

Definition 5.1 (Neural Tangent Kernel)**.** Let $f(x;\theta)\colon \mathbb{R}^d \to \mathbb{R}$ be a neural network with parameters $\theta \in \mathbb{R}^p$. The *neural tangent kernel* (NTK) is the kernel function

$$\Theta(x, x'; \theta) = \big\langle \nabla_\theta f(x;\theta),\ \nabla_\theta f(x';\theta) \big\rangle = \sum_{j=1}^{p} \frac{\partial f(x;\theta)}{\partial \theta_j} \cdot \frac{\partial f(x';\theta)}{\partial \theta_j}, \tag{2}$$

where $\nabla_\theta f(x;\theta) \in \mathbb{R}^p$ is the gradient of the network output with respect to all parameters.

Remark 5.2. The NTK can be viewed as an inner product in a feature space: define $\phi_\theta(x) = \nabla_\theta f(x;\theta) \in \mathbb{R}^p$, so that $\Theta(x, x'; \theta) = \langle \phi_\theta(x), \phi_\theta(x') \rangle$. This is a kernel in the sense of Lesson 61, but with a feature map that depends on the parameters θ.

Example 5.3 (NTK for a Two-Layer Network)**.** Consider a two-layer network with m hidden units:

$$f(x;\theta) = \frac{1}{\sqrt{m}} \sum_{j=1}^{m} a_j \sigma(w_j^\top x),$$

where $\theta = (a_1, \ldots, a_m, w_1, \ldots, w_m)$, the $1/\sqrt{m}$ scaling ensures finite limits as $m \to \infty$, and σ is a smooth activation. The gradients are:

$$\frac{\partial f}{\partial a_j} = \frac{1}{\sqrt{m}} \sigma(w_j^\top x),$$
$$\frac{\partial f}{\partial w_j} = \frac{1}{\sqrt{m}} a_j \sigma'(w_j^\top x)\, x.$$

The NTK is

$$\Theta(x, x'; \theta) = \frac{1}{m} \sum_{j=1}^{m} \sigma(w_j^\top x)\sigma(w_j^\top x') \tag{3}$$
$$+ \frac{1}{m} \sum_{j=1}^{m} a_j^2 \sigma'(w_j^\top x)\sigma'(w_j^\top x')\,(x^\top x'). \tag{4}$$

The first term (3) is the kernel induced by the output-layer gradients; the second term (4) captures the hidden-layer gradients.

5.2 NTK Dynamics of Gradient Descent

Proposition 5.4 (Gradient Descent in Function Space)**.** *Consider training the network $f(x;\theta)$ on data $(x_1, y_1), \ldots, (x_n, y_n)$ by minimizing the squared loss $\mathcal{L}(\theta) = \frac{1}{2}\sum_{i=1}^{n}(f(x_i;\theta) - y_i)^2$ with continuous-time gradient descent $\dot{\theta}(t) = -\nabla_\theta \mathcal{L}(\theta(t))$. Then the network predictions $u_i(t) = f(x_i;\theta(t))$ evolve according to*

$$\dot{u}_i(t) = -\sum_{j=1}^{n} \Theta(x_i, x_j; \theta(t))\,(u_j(t) - y_j), \tag{5}$$

or in matrix form, $\dot{u}(t) = -K(t)(u(t) - y)$, where $K(t) \in \mathbb{R}^{n\times n}$ is the NTK Gram matrix *$K_{ij}(t) = \Theta(x_i, x_j; \theta(t))$.*

Proof. By the chain rule,

$$\dot{u}_i(t) = \frac{d}{dt} f(x_i;\theta(t)) = \nabla_\theta f(x_i;\theta(t))^\top \dot{\theta}(t).$$

The gradient descent dynamics give

$$\dot{\theta}(t) = -\nabla_\theta \mathcal{L}(\theta(t)) = -\sum_{j=1}^{n} (f(x_j;\theta(t)) - y_j)\nabla_\theta f(x_j;\theta(t)).$$

Substituting:

$$\dot{u}_i(t) = -\sum_{j=1}^{n} (u_j(t) - y_j)\nabla_\theta f(x_i;\theta(t))^\top \nabla_\theta f(x_j;\theta(t))$$
$$= -\sum_{j=1}^{n} \Theta(x_i, x_j; \theta(t))(u_j(t) - y_j).$$

□

Remark 5.5. If the NTK Gram matrix $K(t)$ were *constant* (independent of t), then (5) would be a linear ODE with solution $u(t) - y = e^{-Kt}(u(0) - y)$. If K is positive definite, all eigenvalues are positive and $u(t) \to y$ exponentially fast: the training loss converges to zero. This motivates the central question of NTK theory: under what conditions does the NTK remain approximately constant during training?

5.3 The Infinite-Width Limit

Theorem 5.6 (NTK Convergence at Initialization — Jacot et al., 2018)**.** *Consider a fully-connected network of depth L with layer widths $n_1 = n_2 = \cdots = n_{L-1} = m$ (hidden widths all equal to m), smooth activation σ, and random initialization: each weight $W_\ell^{(ij)} \sim \mathcal{N}(0, 2/n_{\ell-1})$ (He initialization). Then as $m \to \infty$:*

(i) The NTK at initialization $\Theta(x, x'; \theta_0)$ converges in probability to a deterministic limit $\Theta_\infty^{(L)}(x, x')$ that depends only on x, x' and the network architecture (depth L and activation σ), not on the random initialization.

(ii) The limiting kernel $\Theta_\infty^{(L)}$ can be computed recursively. Define the NNGP kernel *(neural network Gaussian process kernel) by $\Sigma^{(0)}(x, x') = x^\top x'$ and*

$$\Sigma^{(\ell)}(x, x') = \mathbb{E}_{(u,v)\sim\mathcal{N}(0,\Lambda^{(\ell)})}\big[\sigma(u)\sigma(v)\big], \tag{6}$$

where $\Lambda^{(\ell)} = \left(\begin{smallmatrix} \Sigma^{(\ell-1)}(x,x) & \Sigma^{(\ell-1)}(x,x') \\ \Sigma^{(\ell-1)}(x',x) & \Sigma^{(\ell-1)}(x',x') \end{smallmatrix}\right)$. Then $\Theta_\infty^{(L)}$ is determined by the $\Sigma^{(\ell)}$ kernels and their derivative counterparts $\dot\Sigma^{(\ell)}(x, x') = \mathbb{E}[\sigma'(u)\sigma'(v)]$.

Proof sketch. (i) At initialization, the pre-activations $h_\ell^{(j)} = \sum_k W_\ell^{(jk)} \sigma(h_{\ell-1}^{(k)})$ are sums of m independent random variables (scaled by $1/\sqrt{m}$ via the initialization). By the central limit theorem, as $m \to \infty$, the pre-activations at each layer converge jointly to a Gaussian process. The covariance of this Gaussian process is the NNGP kernel $\Sigma^{(\ell)}$, defined by the recursion (6).

The NTK $\Theta(x, x'; \theta_0) = \sum_{j=1}^p \frac{\partial f(x)}{\partial \theta_j} \frac{\partial f(x')}{\partial \theta_j}$ is also a sum of $O(m)$ terms (one for each parameter). By the law of large numbers (in the $m \to \infty$ limit), this sum converges to its expectation, which is a deterministic function of $\Sigma^{(\ell)}$ and $\dot\Sigma^{(\ell)}$.

(ii) The recursive formula follows from the layer-by-layer structure of the network. The output-layer contribution to the NTK is $\Sigma^{(L-1)}(x, x')$ (from the gradients with respect to W_L), and the hidden-layer contributions involve $\dot\Sigma^{(\ell)}$ (from the chain rule through the activations). The full recursive formula is given in Jacot et al. [2018]. □

Example 5.7 (NTK for a Two-Layer Network with ReLU)**.** For a two-layer ReLU network with He initialization and m hidden units, the limiting NTK as $m \to \infty$ is

$$\Theta_\infty(x, x') = \frac{\|x\|_2 \|x'\|_2}{2\pi}\big(\sin\alpha + (\pi - \alpha)\cos\alpha\big) + \frac{x^\top x'}{2\pi}(\pi - \alpha),$$

where $\alpha = \arccos\big(\frac{x^\top x'}{\|x\|_2\|x'\|_2}\big)$ is the angle between x and x'. This is the "arc-cosine kernel" studied in the kernel methods literature.

5.4 Lazy Training: NTK Stays Constant

Theorem 5.8 (Lazy Training Regime)**.** *Under the same setup as Theorem 5.6, if the width m is sufficiently large (specifically, $m = \Omega(n^4/\lambda_0^4)$ where $\lambda_0 > 0$ is the smallest eigenvalue of the limiting NTK Gram matrix $K_\infty \in \mathbb{R}^{n\times n}$), then with high probability over the random initialization:*

(i) The NTK Gram matrix $K(t)$ stays close to its initial value throughout training: $\|K(t) - K(0)\|_F \le \varepsilon$ for all $t \ge 0$, where $\varepsilon \to 0$ as $m \to \infty$.

(ii) The parameters stay close to their initialization: $\|\theta(t) - \theta_0\|_2 = O(1/\sqrt{m})$ for all t.

(iii) The training loss converges to zero exponentially fast: $\mathcal{L}(\theta(t)) \le e^{-\lambda_0 t}\mathcal{L}(\theta_0)$.

(iv) The network function is approximately linear in the parameters:

$$f(x; \theta(t)) \approx f(x; \theta_0) + \nabla_\theta f(x; \theta_0)^\top (\theta(t) - \theta_0). \tag{7}$$

Proof. **(i) and (ii):** The gradient $\nabla_\theta f(x;\theta)$ is $O(1/\sqrt{m})$ in each coordinate (by the scaling of the initialization). The gradient descent update moves θ by $\Delta\theta = -\eta\nabla_\theta\mathcal{L}(\theta)$, which involves $\nabla_\theta f(x_i;\theta)\cdot(u_i - y_i)$. Since $\nabla_\theta f$ has $O(m)$ coordinates each of size $O(1/\sqrt{m})$, the total movement $\|\theta(t) - \theta_0\|_2 = O(\sqrt{n}/\sqrt{m})$, which is $O(1/\sqrt{m})$ for fixed n.

The NTK involves second-order information: $\Theta(x, x';\theta) - \Theta(x, x';\theta_0)$ is bounded by the Hessian of f times $\|\theta - \theta_0\|$, which is $O(1/\sqrt{m})$ times $O(1/\sqrt{m})$, giving $O(1/m)$ change. This shows the NTK remains approximately constant.

(iii): With $K(t) \approx K(0) \approx K_\infty$, the dynamics (5) become approximately linear: $\dot{u}(t) \approx -K_\infty(u(t) - y)$, with solution $u(t) - y = e^{-K_\infty t}(u(0) - y)$. Since $K_\infty \succeq \lambda_0 I$, we have $\|u(t) - y\|_2^2 \leq e^{-2\lambda_0 t}\|u(0) - y\|_2^2$.

(iv): The linearization (7) follows from Taylor expansion: $f(x;\theta) = f(x;\theta_0) + \nabla_\theta f(x;\theta_0)^\top(\theta - \theta_0) + O(\|\theta - \theta_0\|^2)$, and $\|\theta - \theta_0\|^2 = O(1/m) \to 0$. □

NTK Theory: Training = Kernel Regression

In the infinite-width limit, training a neural network by gradient descent is equivalent to kernel regression with the neural tangent kernel Θ_∞:

1. The NTK converges to a deterministic kernel at initialization.
2. The NTK stays approximately constant during training (lazy training).
3. The network dynamics are approximately linear: $f(x;\theta_t) \approx f(x;\theta_0) + \nabla_\theta f(x;\theta_0)^\top(\theta_t - \theta_0)$.
4. Training converges to zero loss at an exponential rate $e^{-\lambda_0 t}$ determined by the smallest eigenvalue of the NTK Gram matrix.

Example 5.9 (Linearized Network as Random Features)**.** The linearized model (7) can be written as

$$f_{\text{lin}}(x;\alpha) = f(x;\theta_0) + \sum_{j=1}^{p} \alpha_j \frac{\partial f(x;\theta_0)}{\partial \theta_j},$$

where $\alpha = \theta - \theta_0$ are the trainable parameters. This is a *random features model* (Lesson 64, Lesson 64, Section 4.2): the features $\phi_j(x) = \partial f(x;\theta_0)/\partial\theta_j$ are fixed at initialization, and only the linear coefficients α_j are learned. The associated kernel is precisely the NTK: $\Theta(x, x') = \sum_j \phi_j(x)\phi_j(x')$.

We now connect the NTK framework to generalization theory.

6 Implications for Generalization

6.1 NTK Generalization via Kernel Theory

In the lazy training regime (Theorem 5.8), the trained network computes approximately the same function as the kernel regression estimator with kernel Θ_∞. This allows us to transfer the rich generalization theory of kernel methods (Lesson 62) to neural networks.

Proposition 6.1 (Generalization of the NTK Estimator)**.** *In the lazy training regime, the trained network $f(\cdot;\theta_T)$ (after T gradient descent steps or continuous-time training) satisfies the same generalization bounds as kernel ridge regression with kernel Θ_∞ and regularization $\lambda \propto 1/T$. Specifically, if the target function f^* lies in the RKHS $\mathcal{H}_{\Theta_\infty}$ with RKHS norm $\|f^*\|_{\mathcal{H}_{\Theta_\infty}} \leq B$, then the population risk of the trained network satisfies*

$$R(f(\cdot;\theta_T)) - R(f^*) \leq O\left(\frac{B^2}{n} + \frac{B^2}{T}\right).$$

The first term is the statistical error (from n samples) and the second is the optimization error (from T training steps).

Proof sketch. In the lazy regime, $f(\cdot;\theta_T) \approx f_{\text{lin}}(\cdot;\alpha_T)$ where α_T is obtained by gradient descent on the linearized model with features $\nabla_\theta f(\cdot;\theta_0)$. This linearized model is a kernel method with kernel Θ_∞, and its generalization properties follow from standard kernel ridge regression theory (Lesson 62, Section 5). The gradient descent with T steps on the squared loss is equivalent to kernel ridge regression with implicit regularization $\lambda \propto 1/T$ (early stopping regularization). The generalization bound then follows from the bias-variance decomposition for kernel ridge regression. □

Example 6.2 (NTK Generalization for Overparameterized Networks)**.** Consider a two-layer ReLU network with $m = 10{,}000$ hidden units trained on $n = 100$ data points in $\mathbb{R}^{10}$. The network has $p = 10{,}000 \times 11 + 10{,}001 \approx 120{,}000$ parameters ($p \gg n$).

- **VC bound:** $O(\sqrt{p/n}) = O(\sqrt{1200}) \approx 35$ — completely vacuous.
- **NTK bound:** If f^* lies in the NTK RKHS with $\|f^*\|_{\mathcal{H}_\Theta} \le B = 5$, then the excess risk is $O(B^2/n) = O(25/100) = O(0.25)$ — a meaningful guarantee, independent of m.

The NTK framework shows that overparameterization ($m \gg n$) does not hurt generalization, because the trained network implements kernel regression with a fixed kernel.

6.2 Limitations of the NTK Framework

Remark 6.3 (The Feature Learning Gap). The NTK theory has a fundamental limitation: it describes the *lazy training* regime where the network stays close to its initialization and the features $\nabla_\theta f(\cdot;\theta_0)$ remain essentially fixed. This means the network does not *learn features*—it uses the random features from initialization.

In practice, the power of deep learning comes precisely from *feature learning*: the network discovers useful representations in the hidden layers. This happens in the "rich" or "active" training regime, where the parameters move far from initialization and the NTK changes significantly during training. The NTK theory does not capture this regime.

Proposition 6.4 (NTK vs. Finite-Width Networks)**.** *The following gaps exist between the NTK regime and practical neural network training:*

(i) ***Sample complexity.*** *Kernel methods (and hence NTK-regime networks) require $n = \Omega(d^s)$ samples to learn s-smooth functions on $\mathbb{R}^d$, suffering the curse of dimensionality. Finite-width networks with feature learning can potentially avoid this curse (Lesson 65, Lesson 65, Section 6).*

(ii) ***Representation.*** *The RKHS of the NTK may not contain functions that finite-width networks can efficiently represent. The NTK kernel is fixed at initialization and does not adapt to the data, while the effective kernel of a finite-width network evolves during training.*

(iii) ***Depth benefit.*** *In the NTK regime, depth provides only a polynomial (not exponential) improvement in the kernel eigenvalues, while finite-width deep networks exhibit exponential depth separation (Lesson 65, Theorem 4.5).*

Justification. (i) follows from the minimax rates for kernel regression in the RKHS of a fixed kernel; see Bietti and Mairal [2019] for a detailed analysis of NTK eigenvalues and their implications for learning rates.

(ii) The RKHS of the infinite-width NTK is a fixed function space determined by the kernel eigenvalues. Feature learning corresponds to adapting the kernel to the data, which the NTK regime precludes.

(iii) Bietti and Mairal [2019] shows that the eigenvalue decay of the NTK for depth-L networks improves only polynomially with L, while the depth separation results of Telgarsky [2016] show exponential gains for finite-width networks. □

Beyond the NTK in Deep RL

The NTK theory provides a useful theoretical baseline but does not fully explain deep RL success:

- **Feature learning is crucial.** In Atari games and robotic control, the learned representations in hidden layers are essential for good performance. A randomly initialized feature map (as in the NTK regime) would miss task-relevant structure.
- **Non-stationary targets.** In RL, the training targets (Bellman backup targets) change during training, violating the fixed-target assumption of NTK theory.
- **Mean-field theory.** An alternative to the NTK (which linearizes around initialization) is the mean-field theory (Lesson 64, Section 6), which tracks the evolution of the weight distribution and captures feature learning in the large-width limit.

NTK Does Not Explain Deep Learning

While the NTK provides valuable theoretical insights, the consensus in the research community is that the lazy training regime does not capture the behavior of practical deep networks. Zhang et al. [2021] show that standard networks memorize random labels (consistent with the NTK regime) but also generalize on structured data (which requires feature learning beyond the NTK regime). The NTK is best understood as a *limiting case* that is analytically tractable but does not capture the full power of deep learning.

7 Exercises

Exercise 7.1 (Rademacher Complexity of Bounded Linear Functions). (*)
Let $\mathcal{F} = \{x \mapsto w^\top x : \|w\|_1 \le B\}$ and $\|x_i\|_\infty \le C$ for all i. Show that $\hat{\mathrm{Rad}}_S(\mathcal{F}) \le BC\sqrt{2\log(2d)/n}$.

Hint: Use $\sup_{\|w\|_1 \le B} w^\top z = B\|z\|_\infty$ and the fact that $\mathbb{E}[\max_{j\le d} Z_j] \le \sqrt{2\log d}\cdot\max_j(\mathbb{E}[Z_j^2])^{1/2}$ for sub-Gaussian Z_j.

Exercise 7.2 (Computing the NTK for a Simple Network). (*)
Consider a two-layer network $f(x; a, w) = a\sigma(wx + b)$ with scalar input $x \in \mathbb{R}$, parameters $\theta = (a, w, b)$, and smooth activation σ. Compute the NTK $\Theta(x, x'; \theta) = \nabla_\theta f(x)^\top \nabla_\theta f(x')$ explicitly.

Exercise 7.3 (Spectral Norm Bound for a Specific Network). (**)
Consider a 3-layer ReLU network on $\mathbb{R}^{10}$ with layer widths $(10, 50, 50, 1)$ and weight matrices W_1, W_2, W_3.

(a) Suppose $\|W_1\|_2 = 1.5$, $\|W_2\|_2 = 1.2$, $\|W_3\|_2 = 0.8$, and $B_x = 1$. Compute the spectrally-normalized Rademacher complexity bound from Theorem 4.3 (ignoring the logarithmic factor).

(b) How many training samples n are needed for the generalization gap to be at most 0.1?

(c) The network has $10 \times 50 + 50 \times 50 + 50 \times 1 = 3{,}050$ parameters. Compare the VC-based sample complexity with the norm-based sample complexity from part (b).

Exercise 7.4 (NTK Gram Matrix and Training Convergence). (**)
Consider training data $(x_1, y_1), (x_2, y_2)$ with the NTK Gram matrix $K = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$.

(a) Compute the eigenvalues of K and verify that K is positive definite.

(b) Using the continuous-time gradient descent solution $u(t) - y = e^{-Kt}(u(0) - y)$, compute $u(t)$ for $u(0) = (0, 0)^\top$ and $y = (1, -1)^\top$.

(c) How long does it take for the training loss $\|u(t)-y\|_2^2$ to decrease to 1% of its initial value?

(d) If the NTK Gram matrix were $K' = \begin{pmatrix} 2 & 1.99 \\ 1.99 & 2 \end{pmatrix}$, how would the convergence rate change? Relate this to the condition number.

Exercise 7.5 (Rademacher Complexity and Value Function Generalization)**.** (**)
In fitted Q-iteration, the Q-function is approximated by a 3-layer ReLU network with spectral norms $\|W_\ell\|_2 \le s$ for each layer. States satisfy $\|s\|_2 \le 1$ and rewards are bounded by 1.

(a) Using Theorem 4.3, bound the Rademacher complexity of the Q-network class.

(b) Apply Theorem 3.6 to bound the generalization gap of each Bellman backup step.

(c) If K backups are performed and the discount factor is γ, bound the total error of fitted Q-iteration.

Exercise 7.6 (NTK Feature Learning Gap)**.** (***)
Consider learning the function $f^*(x) = \|Ax\|_2$ on $\mathbb{R}^d$ where $A \in \mathbb{R}^{k\times d}$ is an unknown rank-k matrix with $k \ll d$.

(a) Show that f^* lies in a k-dimensional subspace: there exists a feature map $\phi(x) = Ax \in \mathbb{R}^k$ such that f^* depends on x only through $\phi(x)$.

(b) Argue that a neural network that learns the features ϕ (feature learning regime) needs $O(k)$ effective parameters to represent f^*, while the NTK regime (random features) requires $O(d^s)$ parameters for s-smooth functions.

(c) Conclude that for $k \ll d$, the feature learning regime is exponentially more sample-efficient than the NTK regime for this target function.

8 Summary

- **Rademacher complexity** $\mathrm{Rad}_n(\mathcal{F}) = \mathbb{E}[\sup_{f\in\mathcal{F}} \frac{1}{n}\sum_i \varepsilon_i f(x_i)]$ measures how well a function class can correlate with random noise. It provides a more refined capacity measure than VC dimension, accounting for the data distribution and real-valued functions.
- The **fundamental Rademacher bound**: the uniform deviation $\sup_f |R(f) - \hat{R}_n(f)|$ is bounded by $2\mathrm{Rad}_n(\mathcal{F}) + O(\sqrt{\log(1/\delta)/n})$ with high probability.
- For deep ReLU networks, the **spectrally-normalized bound** gives $\mathrm{Rad}_n \le O(B_x \prod_\ell \|W_\ell\|_2 \cdot \sqrt{L\log W}/\sqrt{n})$. This depends on **weight norms, not parameter counts**, explaining why overparameterized networks can generalize.
- The **neural tangent kernel** $\Theta(x,x') = \langle \nabla_\theta f(x), \nabla_\theta f(x')\rangle$ governs the dynamics of gradient descent on neural networks: $\dot{u}(t) = -K(t)(u(t)-y)$ where $K_{ij}(t) = \Theta(x_i, x_j; \theta(t))$.
- In the **infinite-width limit**: (1) the NTK converges to a deterministic kernel Θ_∞, (2) the NTK stays approximately constant during training (lazy training), (3) the network dynamics are linear, and (4) training converges exponentially fast.
- The **lazy training regime** shows that training an overparameterized network is equivalent to kernel regression with the NTK, allowing generalization guarantees to be transferred from kernel theory.
- **Limitations**: the NTK regime does not capture *feature learning*, which is where deep networks truly excel. The NTK gives curse-of-dimensionality rates, while practical networks with feature learning can exploit structure to achieve faster rates.

- The norm-based Rademacher bound and the NTK theory provide complementary perspectives on generalization: the former says "small weights imply generalization," and the latter says "overparameterized networks implement kernel regression."

Neural Network Theory Subdomain Summary

This capstone lesson completes the Neural Network Theory subdomain (Lessons 59–66). The following table summarizes the arc of the subdomain.

Lesson	Topic	Key Result	Theme
59	ERM and Generalization	VC dimension bounds; fundamental theorem of statistical learning	Statistical learning
60	Bias-Variance and Model Selection	Bias-variance decomposition; oracle inequalities for model selection	Tradeoff analysis
61	Kernel Methods	Mercer's theorem; the kernel trick for implicit feature maps	Feature spaces
62	RKHS and Representer Theorem	Representer theorem reduces infinite-dimensional optimization to finite	Regularization
63	Universal Approximation	Cybenko/Hornik: non-polynomial activations yield density in $C(K)$	Expressiveness
64	Barron's Theorem	Dimension-free $O(1/m)$ rate for Barron-class functions	Approximation rates
65	Depth-Width Tradeoffs	Telgarsky: depth L needs width $2^{\Omega(L)}$ in shallow nets; compositional gains	Architecture
66	NTK and Rademacher Complexity	Norm-based generalization; infinite-width networks = kernel regression	Generalization

The subdomain progresses from the foundations of statistical learning (what does it mean to generalize?) through kernel methods (a tractable infinite-dimensional framework) to neural network-specific results (universal approximation, approximation rates, depth advantages, and overparameterization). The key tension throughout is between *expressiveness* (can the network represent the target?) and *generalization* (does it learn the target from finite data?). Barron's theorem and the NTK provide two different resolutions: Barron shows dimension-free approximation for a restricted class, while the NTK shows that overparameterization does not hurt generalization (in the lazy regime).

9 Further Reading

- **Lesson 67** (RNNs and BPTT) begins the next subdomain (3.2: Recurrent Architectures), studying how temporal depth interacts with the expressiveness and trainability results developed here.
- **Phase 04** (Deep RL) applies the theory of this subdomain to RL-specific settings: DQN and its extensions (Lesson 74), trust region methods (Lesson 76), PPO (Lesson 77), and SAC (Lesson 79).
- Golowich et al. [2018] improves the spectrally-normalized bound by removing the dependence on depth in the logarithmic factor, showing that the bound can be made independent of both depth and width.

- Allen-Zhu et al. [2019] provides convergence guarantees for gradient descent on overparameterized networks that go beyond the NTK regime, using a notion of "semi-smoothness" of the loss landscape.
- Chizat and Bach [2018] develops the mean-field theory of two-layer networks as an alternative to the NTK: the "active" or "rich" regime where features evolve during training and the network does not stay close to initialization.
- Zhang et al. [2021] presents the celebrated experiments showing that overparameterized networks can memorize random labels yet generalize on structured data, demonstrating that classical generalization theory (VC, Rademacher) alone cannot explain deep learning generalization.
- **Historical note.** The NTK was introduced by Jacot et al. [2018] in 2018, building on earlier observations by Neal (1996) that wide neural networks converge to Gaussian processes. The connection between overparameterization and kernel methods was independently developed by Du et al. [2019] and Allen-Zhu et al. [2019]. The spectrally- normalized bound of Bartlett et al. [2017] was motivated by the practical success of spectral normalization in GANs and the theoretical need for parameter-count-independent generalization bounds.

References

Zeyuan Allen-Zhu, Yuanzhi Li, and Zhao Song. A convergence theory for deep learning via over-parameterization. *Proceedings of the 36th International Conference on Machine Learning*, pages 242–252, 2019.

Sanjeev Arora, Simon S. Du, Wei Hu, Zhiyuan Li, and Ruosong Wang. Fine-grained analysis of optimization and generalization for overparameterized two-layer neural networks. *Proceedings of the 36th International Conference on Machine Learning*, pages 322–332, 2019.

Peter L. Bartlett and Shahar Mendelson. Rademacher and Gaussian complexities: Risk bounds and structural results. *Journal of Machine Learning Research*, 3:463–482, 2002.

Peter L. Bartlett, Dylan J. Foster, and Matus J. Telgarsky. Spectrally-normalized margin bounds for neural networks. *Advances in Neural Information Processing Systems*, 30:6240–6249, 2017.

Alberto Bietti and Julien Mairal. On the inductive bias of neural tangent kernels. *Advances in Neural Information Processing Systems*, 32, 2019.

Lénaïc Chizat and Francis Bach. On the global convergence of gradient descent for over-parameterized models using optimal transport. *Advances in Neural Information Processing Systems*, 31:3036–3046, 2018.

Simon S. Du, Xiyu Zhai, Barnabás Póczos, and Aarti Singh. Gradient descent provably optimizes over-parameterized neural networks. *International Conference on Learning Representations*, 2019.

Noah Golowich, Alexander Rakhlin, and Ohad Shamir. Size-independent sample complexity of neural networks. *Conference on Learning Theory*, pages 297–299, 2018.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in Neural Information Processing Systems*, 31: 8571–8580, 2018.

Michel Ledoux and Michel Talagrand. Probability in Banach spaces. 1991.

Jaehoon Lee, Lechao Xiao, Samuel S. Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. *Advances in Neural Information Processing Systems*, 32, 2019.

Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. Norm-based capacity control in neural networks. *Conference on Learning Theory*, pages 1376–1401, 2015.

Matus Telgarsky. Benefits of depth in neural networks. *Conference on Learning Theory*, pages 1517–1539, 2016.

Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3): 107–115, 2021.